

Lecture 3: an outline

- ▶ Extending matrix manipulations:
 - ▶ Occurrence of multidimensional arrays.
 - ▶ Operation with them.
- ▶ Multidimensional differentiation:
 - ▶ Chain rule.
 - ▶ Linear algebraic treatment.

Lecture 3: Multidimensional arrays: examples, explanation

Examples of data in this format:

- ▶ An RGB - coded picture P of size $\mathbb{R}^{1920 \times 1080}$.
 - ▶ Here $P[i, j, k]$ is the intensity of the pixel point (i, j) regarding the k th color ($k \in \{1, 2, 3\}$).
 - ▶ Here we have an array of size $1920 \times 1080 \times 3$.
 - ▶ Three layers of matrices corresponding to the amount of red, green, and blue in each pixel.
- ▶ Maximum daily temperature T in the past 10 years measured at grid of size 40×50 .
 - ▶ Here $T \in \mathbb{R}^{3652 \times 40 \times 50}$.
 - ▶ This can be recognized as 3652 pieces of matrices with size 40×50 .

Lecture 3: Multidimensional arrays; examples (continued)

First example can be extended - really real life case:

- ▶ A number of 10000 RGB - coded picture P of size $\mathbb{R}^{1920 \times 1080}$.
 - ▶ Can be represented with a four-dimensional array Q .
 - ▶ $Q \in \mathbb{R}^{10000 \times 1920 \times 1080 \times 3}$.
 - ▶ Could also be: $Q \in \mathbb{R}^{1920 \times 1080 \times 3 \times 10000}$.

Lecture 3: Arrays in Python

- ▶ Again, in Python:

arrays of size $1 \times j \times k \not\equiv$ arrays of size $j \times k$.

- ▶ At the same time, we can “reshape” $Q \in \mathbb{R}^{j \times k}$ to have $\tilde{Q} \in \mathbb{R}^{1 \times j \times k}$ or the other way around.
- ▶ Also, we have array operations similarly to the matrix-case.
- ▶ Elementwise addition, multiplication and basic operations work in the same way.
 - Why is this important?
 - ▶ Applying a long loop is slower compared to a built-in vectorized operation.
 - ▶ Frequently, we should rescale (“normalize”) the entire data set (e.g., divide with the maximum).

Lecture 3: Arrays in Python: multiplication

- ▶ Matrix-matrix product can be generalized using `matmul`.
 - ▶ Here operation is performed in the last components.
 - ▶ In concrete terms, having an array of dimension (d_1, d_2, d_3) can be multiplied
 - ▶ with an array of size (d_1, d_3, d_4) to obtain an array of dimension (d_1, d_2, d_4) – matrix-matrix product for d_1 pair of matrices
 - ▶ with a matrix of size (d_3, d_4) to obtain an array of dimension (d_1, d_2, d_4) – matrix-matrix product with a fixed second component,
 - ▶ with a vector of size d_4 to obtain a matrix of dimension (d_1, d_2) – matrix-vector product with a fixed second component,.

Lecture 3: Arrays in Python: reshape, "flatten"

- ▶ Having an array of size $d_1 \times d_2 \times d_3$, you can put these elements into *any* array having the same number of elements.
 - ▶ In particular: you can "flatten out" any array row-wise, beginning with the first row (of the first matrix) and taking the second row (of the first matrix)
 - ▶ You can change this order.
 - ▶ This is also a linear map.

Could it be represented with a matrix? Or with an array?

Lecture 3: one- and multidimensional chain rule

- ▶ For given differentiable functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$

$$x \xrightarrow{g} g(x) \xrightarrow{f} f(g(x)), \quad (f \circ g)'(x) = f'(g(x)) \cdot g'(x)$$

provided that the right-hand side makes sense.

- ▶ Easy example: $[(2x + 5)^3]'$; here: $g(x) = 2x + 5, f(z) = z^3$.

- ▶ Therefore, we get $[(2x + 5)^3]' = 3 \cdot (2x + 5)^2 \cdot 2$.

- ▶ Multidimensional case:

$$\mathbb{R}^n \xrightarrow{g} \mathbb{R}^k \xrightarrow{f} \mathbb{R}^m.$$

- ▶ Instead of $\mathbf{f}', \mathbf{g}'$ we frequently write $\nabla \mathbf{f}$ or $\nabla \mathbf{g}$.

- ▶ If $\mathbf{f} = [f_1, f_2, \dots, f_m]$, we have

$$\nabla \mathbf{f} = [[\nabla f_1], [\nabla f_2], \dots, [\nabla f_m]] \in \mathbb{R}^{m \times k}.$$

Lecture 3: multidimensional chain rule - basics

- ▶ Formally, we have the same rule:

$$(\mathbf{f} \circ \mathbf{g})'(\mathbf{x}) = \mathbf{f}'(\mathbf{g}(\mathbf{x})) \cdot \mathbf{g}'(\mathbf{x}),$$

provided that the right hand side makes sense, but here

- ▶ $\mathbf{f}' : \mathbb{R}^k \rightarrow \mathbb{R}^{m \times k}$, i.e. $\mathbf{f}'(\mathbf{g}(\mathbf{x})) \in \mathbb{R}^{m \times k}$
- ▶ $\mathbf{g}' : \mathbb{R}^n \rightarrow \mathbb{R}^{k \times n}$, i.e. $\mathbf{g}'(\mathbf{x}) \in \mathbb{R}^{k \times n}$.
- ▶ Therefore, application of chain rule: computation of matrix products.
 - ▶ In concrete terms: $(\mathbf{f} \circ \mathbf{g})'(\mathbf{x}) \in \mathbb{R}^{m \times n}$.
- ▶ We extend the above concept to the function of form $\psi \circ f_N \circ f_{N-1} \circ \cdots \circ f_2 \circ f_1$.

Lecture 3: multidimensional chain rule - real-life case

- ▶ We have a number of consecutive steps:

$$\mathbb{R}^{d_0} \xrightarrow{\mathbf{f}_1} \mathbb{R}^{d_1} \xrightarrow{\mathbf{f}_2} \mathbb{R}^{d_2} \dots \mathbb{R}^{d_{N-1}} \xrightarrow{\mathbf{f}_N} \mathbb{R}^{d_N} \xrightarrow{\psi} \mathbb{R}.$$

- ▶ $\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_N$ are mappings between consecutive layers,
- ▶ ψ is a (real-valued) loss function.

- ▶ Then the derivative of $\psi \circ \mathbf{f}_N \circ \mathbf{f}_{N-1} \circ \dots \circ \mathbf{f}_2 \circ \mathbf{f}_1$ in $\mathbf{x}$:

$$\psi'(\mathbf{f}_N \circ \dots \circ \mathbf{f}_2 \circ \mathbf{f}_1(\mathbf{x})) \cdot \mathbf{f}'_N(\mathbf{f}_{N-1} \circ \dots \circ \mathbf{f}_2 \circ \mathbf{f}_1(\mathbf{x})) \cdot \dots \cdot \mathbf{f}'_1(\mathbf{x})$$

- ▶ The first term is a vector, the rest are matrices.
- ▶ The result is a vector.

Lecture 3: multidimensional chain rule: final remarks

- ▶ Do not be afraid: it is done also by the computer.
 - ▶ The main tool for this: backward differentiation algorithm.
 - ▶ Be careful: if operations (functions) are not differentiable, the algorithm may not work.
 - ▶ There are remedies, exceptions and depends on Python versions.

Lecture 4: an outline

- ▶ Going on with special matrices and their properties:
 - ▶ range of matrices,
 - ▶ projections,
- ▶ permutations.

Lecture 4: Back to the geometry of $\mathbb{R}^k$ – subspaces

- ▶ A set $S \subset \mathbb{R}^k$ is called a subspace if:
 - ▶ For all $\mathbf{u}, \mathbf{v} \in S$: $\mathbf{u} + \mathbf{v} \in S$,
 - ▶ for all $\lambda \in \mathbb{R}, \mathbf{u} \in S$: $\lambda \mathbf{u} \in S$.
- ▶ Each subspace can be “generated” by vectors $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$ with $n < k$ in the sense that for all $\mathbf{u} \in S$:
 $\mathbf{u} = a_1 \mathbf{b}_1 + \dots + a_n \mathbf{b}_n$, where the coefficients are unique.
 - ▶ Here n is called the dimension of the subspace.
 - ▶ $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$ is called a *basis*: a maximal independent system in S .
- ▶ “Visible” example: a plane in $\mathbb{R}^3$.
 - ▶ Here $k = 3, n = 2$.
 - ▶ Basis vectors can be two any independent vectors in that plane.

Lecture 4: Back to the geometry of $\mathbb{R}^k$ – orthogonality

- ▶ The length of a vector $\mathbf{v} \in \mathbb{R}^k$ is given by

$$|\mathbf{v}| = \sqrt{\mathbf{v} \cdot \mathbf{v}}$$

- ▶ Vectors $\mathbf{v} \in \mathbb{R}^k$ and $\mathbf{u} \in \mathbb{R}^k$ are called *orthogonal* if $\mathbf{v} \cdot \mathbf{u} = 0$.

- ▶ Notation: $\mathbf{v} \perp \mathbf{u}$.

- ▶ We can orthogonally decompose $\mathbf{v}$ with respect to $\mathbf{u}$ as:

$$\mathbf{v} = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}|^2} \mathbf{u} + \mathbf{v} - \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}|^2} \mathbf{u} \quad (\mathbf{u}\text{-orthogonal component})$$

- ▶ A subspace $S \subset \mathbb{R}^k$ is called orthogonal to $\mathbf{v} \in \mathbb{R}^k$ if for all $\mathbf{u} \in S$ we have $\mathbf{v} \perp \mathbf{u}$.

- ▶ In the above situation:

S is orthogonal to $\mathbf{v} \Leftrightarrow S$ is orthogonal to $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k$.

Lecture 4: Range and rank of matrices

- ▶ Motivation: "compressing" large matrices.
 - ▶ Represent them with a smaller one (we did it in Lecture 2).
- ▶ Visible example: Each vector in $\mathbb{R}^3$ we project (orthogonally) to $(1, 0, 1)^T$; notation: P .
 - ▶ $P((1, 0, 0)^T) = (\frac{1}{2}, 0, \frac{1}{2})$, $P((0, 0, 1)^T) = (\frac{1}{2}, 0, \frac{1}{2})$,
 $P((0, 1, 0)^T) = (0, 0, 0)$.
 - ▶ The corresponding matrix: $P = \begin{pmatrix} \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \end{pmatrix}$
- ▶ It would be better to represent P only with $(1, 0, 1)^T$.

Lecture 4: Range and rank of matrices

- ▶ Rank of a matrix B :
 - ▶ The number of its independent row vectors.
 - ▶ The number of its independent column vectors.
 - ▶ The dimension of the subspace containing all row vectors.
 - ▶ The dimension of the subspace containing all column vectors.
- ▶ They are all the same.
- ▶ We can represent $B \in \mathbb{R}^{n \times k}$ with a matrix $B \in \mathbb{R}^{\text{rank } B \times k}$.

Lecture 4: Projection matrices

- ▶ Example; construction in case of a 3-dimensional range:
 - ▶ Take a basis $\{\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3\}$ in $S \subset \mathbb{R}^n$.
 - ▶ Take the projection of the original basis elements to $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$.
 - ▶ This can be given with 3 numbers.
 - ▶ Summarized; such a projection can be given with $P \in \mathbb{R}^{3 \times n}$.
- ▶ How to get $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ in practice?

Lecture 4: Stochastic matrices

- ▶ A class of matrices with
 - ▶ Non-negative elements.
 - ▶ Sum of the elements in rows is one.
- ▶ They need to be square-matrices.
- ▶ A special sub-class I: doubly stochastic matrices:
 - ▶ Also, sum of the elements in columns is one.
- ▶ A special sub-sub-class: Permutation matrices.
 - ▶ A permutation of n elements can be given by the image of all elements.
 - ▶ $P_{j,k} = 1 \Leftrightarrow j\text{th element is sent to the } k\text{th place.}$
 - ▶ Multiplied with a vector: permutes its components accordingly.

Lecture 4: Stochastic matrices -examples

▶ A general stochastic matrix:

$$\begin{pmatrix} 0.2 & 0.2 & 0 & 0.6 \\ 0.2 & 0.4 & 0 & 0.4 \\ 0.2 & 0.2 & 0.3 & 0.3 \\ 0.3 & 0.2 & 0.3 & 0.2 \end{pmatrix}$$

▶ A doubly stochastic matrix:

$$\begin{pmatrix} 0.2 & 0.2 & 0 & 0.6 \\ 0.2 & 0.4 & 0.4 & 0.0 \\ 0.2 & 0.2 & 0.3 & 0.3 \\ 0.4 & 0.2 & 0.3 & 0.1 \end{pmatrix}$$

▶ A permutation matrix: $P =$

$$\begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

▶ $P[w_1, w_2, w_3, w_4]^T = [w_2, w_1, w_4, w_3]^T$